

Code Complexity Analysis – DirectedLineUpgradePlan (Kosaraju & Topological Sort)

Table of Contents

1	Pseudocode	2
1.1	Function 1: computeUpgradePlan	2
1.2	Function 2: kosarajuAlgorithm	2
1.3	Function 3: computeFinishTimes	2
1.4	Function 4: findSCCsAndCycle	3
1.5	Function 5: DFSFinishTime	3
1.6	Function 6: reverseGraph	3
1.7	Function 7: getWeaklyConnectedComponents	4
1.8	Function 8: buildUndirectedGraph	4
1.9	Function 9: reconstructWCCGraph	4
1.10	Function 10: dfs	5
2	Time Complexity	6
2.1	Function: computeUpgradePlan	6
2.2	Function: kosarajuAlgorithm	6
2.3	Function: computeFinishTimes	6
2.4	Function: findSCCsAndCycle	7
2.5	Function: getWeaklyConnectedComponents	7
2.6	Function: buildUndirectedGraph	8
2.7	Function: reconstructWCCGraph	9
2.8	Function: DFSFinishTime	9
2.9	Function: reverseGraph	9
2.10	Function: dfs	10
3	DFS Time Complexity Analysis	11

1 Pseudocode

1.1 Function 1: computeUpgradePlan

Location: DirectedLineUpgradePlan Implementation

```
1 IF graph is null OR graph is not directed
2   THROW IllegalArgumentException
3
4 WCCs = CALL getWeaklyConnectedComponents(graph)
5 INITIALIZE finalTopologicalOrders as List of Lists
6
7 FOR EACH wcc IN WCCs
8   INITIALIZE finishStack as LinkedList
9   INITIALIZE cycleResult as LinkedList
10
11   CALL kosarajuAlgorithm(wcc, cycleResult, finishStack)
12
13   IF cycleResult IS NOT EMPTY
14     RETURN new UpgradePlanResult(true, null, cycleResult)
15
16   REVERSE finishStack
17   ADD finishStack TO finalTopologicalOrders
18
19 RETURN new UpgradePlanResult(false, finalTopologicalOrders, null)
```

1.2 Function 2: kosarajuAlgorithm

Location: DirectedLineUpgradePlan Implementation

```
1 INITIALIZE visited as Boolean Array [graph.numVertices]
2 CALL computeFinishTimes(graph, visited, finishStack)
3
4 tGraph = CALL reverseGraph(graph)
5 CALL findSCCsAndCycle(tGraph, finishStack, cycle, visited)
```

1.3 Function 3: computeFinishTimes

Location: DirectedLineUpgradePlan Implementation

```
1 FOR EACH vertex v IN graph.vertices
2   IF visited[graph.key(v)] IS false
3     CALL DFSFinishTime(graph, v, visited, finishStack)
```

1.4 Function 4: findSCCsAndCycle

Location: DirectedLineUpgradePlan Implementation

```
1 INITIALIZE visited as Boolean Array [tGraph.numVertices]
2 INITIALIZE tempStack as Copy of finishStack
3
4 WHILE tempStack IS NOT EMPTY
5     v = tempStack.pop() // remove last
6     IF visited[tGraph.key(v)] IS false
7         INITIALIZE SCC as LinkedList
8         CALL DFSFinishTime(tGraph, v, visited, SCC)
9
10    IF SCC.size > 1
11        ADD ALL SCC nodes TO cycle
12    RETURN
```

1.5 Function 5: DFSFinishTime

Location: DirectedLineUpgradePlan Implementation

```
1 vKey = g.key(vOrig)
2 visited[vKey] = true
3
4 FOR EACH neighbor IN g.adjVertices(vOrig)
5     neighborKey = g.key(neighbor)
6     IF visited[neighborKey] IS false
7         CALL DFSFinishTime(g, neighbor, visited, qdfs)
8
9 ADD vOrig TO qdfs // Tempo de finaliza o
```

1.6 Function 6: reverseGraph

Location: DirectedLineUpgradePlan Implementation

```
1 CREATE tGraph AS NEW MapGraph(directed=true)
2
3 FOR EACH v IN graph.vertices
4     tGraph.addVertex(v)
5
6 FOR EACH edge IN graph.edges
7     tGraph.addEdge(edge.dest, edge.origin, edge.weight)
8
9 RETURN tGraph
```

1.7 Function 7: getWeaklyConnectedComponents

Location: DirectedLineUpgradePlan Implementation

```
1 undirectedGraph = CALL buildUndirectedGraph(graph)
2 INITIALIZE visited as Boolean Array
3 INITIALIZE wccs as List of Graphs
4
5 FOR EACH vertex v IN undirectedGraph
6     IF visited[v.key] IS false
7         INITIALIZE componentVertices as Set
8         CALL dfs(undirectedGraph, v, visited, componentVertices)
9
10        wccGraph = CALL reconstructWCCGraph(graph, componentVertices)
11        ADD wccGraph TO wccs
12
13 RETURN wccs
```

1.8 Function 8: buildUndirectedGraph

Location: DirectedLineUpgradePlan Implementation

```
1 CREATE undirectedGraph AS NEW MapGraph(directed=false)
2
3 FOR EACH v IN graph.vertices
4     undirectedGraph.addVertex(v)
5
6 FOR EACH edge IN graph.edges
7     undirectedGraph.addEdge(edge.origin, edge.dest, edge.weight)
8
9 RETURN undirectedGraph
```

1.9 Function 9: reconstructWCCGraph

Location: DirectedLineUpgradePlan Implementation

```
1 CREATE wccGraph AS NEW MapGraph(directed=true)
2
3 FOR EACH cv IN componentVertices
4     wccGraph.addVertex(cv)
5
6 FOR EACH edge IN originalGraph.edges
7     IF componentVertices CONTAINS edge.origin AND componentVertices CONTAINS
   edge.dest
8         wccGraph.addEdge(edge.origin, edge.dest, edge.weight)
9
10 RETURN wccGraph
```

1.10 Function 10: dfs

Location: DirectedLineUpgradePlan Implementation

```
1 vKey = graph.key(v)
2 visited[vKey] = true
3 ADD v TO component
4
5 FOR EACH neighbor IN graph.adjVertices(v)
6     IF visited[graph.key(neighbor)] IS false
7         CALL dfs(graph, neighbor, visited, component)
```

2 Time Complexity

2.1 Function: computeUpgradePlan

Operation	Description	Time Complexity	Notes
IF graph is null...	Throw exception if null/not directed	1	Constant
CALL getWeaklyConnect	Compute WCCs	$V + E$	
INITIALIZE finalOrders		1	
FOR EACH wcc IN WCCs	Iterate all components	k	k is the number of WCCs.
new LinkedList()	Initialize finishStack & cycleResult	$2k$	
kosarajuAlgorithm(cycleResult, finishStack)	Find SCCs and cycle in component	$V + E$	
IF cycleResult NOT EMPTY	Check for cycles	k	
REVERSE finishStack	Reverse topological order	$V * k$	
ADD finishStack TO finalTopological	Store topological order	$V * k$	
Total		$2V + 2E + 2V * k + 4k + 2$	
Big(O)		$O(V + E)$	

2.2 Function: kosarajuAlgorithm

Operation	Description	Time Complexity	Notes
INITIALIZE visited	Create visited array	V	
CALL computeFinishTimes(visited, finishStack)	First DFS to compute finish times	$V + E$	
tGraph = CALL reverseGraph	Transpose the graph	$\sum V_i + E_i = V + E$	
findSCCsAndCycle(finishStack, cycle, visited)	Second DFS to find SCCs and detect cycle	$V + E$	
Total		$4V + 3E$	
Big(O)		$O(V + E)$	

2.3 Function: computeFinishTimes

Operation	Description	Time Complexity	Notes
FOR EACH vertex v	Iterate all vertices	V	
<code>graph.key(v)</code>	Get vertex by index	V	
IF <code>visited[...]</code> IS false	Check if vertex visited	V	
<code>DFSFinishTime(g, v, visited, finishStack)</code>	DFS starting at unvisited vertex	$\sum V_i + E_i = V + E$	Total over all calls
Total		$4V + E$	
Big(O)		$O(V + E)$	

2.4 Function: findSCCsAndCycle

Operation	Description	Time Complexity	Notes
INITIALIZE <code>visited</code>	Reset visited array	V	
INITIALIZE <code>tempStack</code>	Copy finish stack	V	
WHILE <code>tempStack</code> NOT empty	Iterate all vertices	V	Each vertex popped once
<code>tempStack.remove</code>	Remove vertex from stack	V	$O(1)$ per pop
IF <code>visited[...]</code> IS false	Check if visited	V	$O(1)$
INITIALIZE SCC	Create SCC list	k	Created once per SCC (k).
<code>DFSFinishTime(t, v, visited, SCC)</code>	DFS on transposed graph	$\sum V_i + E_i = V + E$	
IF <code>SCC.size > 1</code>	Check component size	k	
<code>cycle.addAll(SCC)</code>	Add SCC to cycle list	V	Only once (early return). In the worst case, all nodes are added.
Total		$7V + 2k + E$	Dominant terms
Big(O)		$O(V + E)$	

2.5 Function: getWeaklyConnectedComponents

Operation	Description	Time Complexity	Notes
CALL buildUndirectedGraph	Convert directed to undirected graph	$4V + 3E + 1$	
INITIALIZE visited	Create visited array	V	Initialized to false
INITIALIZE wccs	Create list of components	1	
FOR EACH vertex v	Iterate all vertices	V	
undirectedGraph. getVertex(v)	Get vertex by index	V	
IF visited[v.key] IS false	Check visited	V	$O(1)$
INITIALIZE componentVertices set		k	
CALL dfs(undirectedGraph, v, ...)	DFS traversal	$\sum V_i + E_i = V + E$	Total over all calls $O(V + E)$
CALL reconstructWCCGraph componentVertices	Build WCC subgraph	$V + E$	
wccs.add(wccGraph)	Add component to list	k	One per component
Total		$7V + 5E + 2k + 2$	Dominant terms
Big(O)		$O(V + E)$	

Note

$$\sum(E * V_i) = E * \sum(V_i) = E * V$$

2.6 Function: buildUndirectedGraph

Operation	Description	Time Complexity	Notes
CREATE undirectedGraph	Create empty undirected graph	1	Constant
graph.vertices()		V	
FOR EACH v IN graph.vertices	Iterate all vertices	V	
undirectedGraph. addVertex(v)	Add vertex to graph	V	
graph.edges()		$V + E$	
FOR EACH edge	Iterate all edges	E	
undirectedGraph. addEdge(edge.src, edge.dest, edge.weight)	Add edge to graph	E	
Total		$4V + 3E + 1$	
Big(O)		$O(V + E)$	

2.7 Function: reconstructWCCGraph

Operation	Description	Time Complexity	Notes
CREATE wccGraph	Create empty graph	1	Constant
FOR EACH cv IN componentVertices	Iterate component vertices	V_i	V_i is the number of vertices in the WCC.
wccGraph.addVertex	Add vertex to sub-graph	V_i	
originalGraph.edges	Get all edges	$V + E$	This retrieves edges from the whole graph.
FOR EACH edge	Iterate all edges	E	
componentVertices	Check origin in component	1	
componentVertices	Check destination in component	1	
wccGraph.addEdge	Add edge to graph	E_i	
Total		$V + 2E + E_i + 1$	
Big(O)		$O(V + E)$	Dominant term

2.8 Function: DFSFinishTime

Operation	Description	Time Complexity	Notes
vKey = g.key(vOrig)	Get vertice key	V_i	
visited[vKey] = true	Mark the current vertex as visited	V_i	
FOR EACH neighbor	Iterate over all neighbors(edges)	E_i	$\deg(v) = E$ (directed graph)
neighborKey = g.key(neighbor)	Get index/key of neighbor	E_i	
IF visited[neighbor] IS false	Check if neighbor is visited	E_i	
CALL DFSFinishTime(...)	Recursive DFS call	E_i	
ADD vOrig TO qdfs	Add current vertex to finish stack	V_i	
Total		$3V_i + 4E_i$	
Big(O)		$O(V_i + E_i)$	

2.9 Function: reverseGraph

Operation	Description	Time Complexity	Notes
new MapGraph<>(graph)	Create empty MapGraph	1	
graph.vertices()	Get all vertices	V_i	
for (StationVertex v : graph.vertices()) t.graph.addVertex(v)	Add all vertices	V_i	
graph.edges()	Get all edges	$V_i + E_i$	
for (Edge<StationVertex> edge : edges) tGraph.addEdge(edge)	Add reversed edges	E_i	
return tGraph	Return graph	1	
Total		$3V_i + 2E_i + 2$	
Big(O)		$O(V_i + E_i)$	

2.10 Function: dfs

Operation	Description	Time Complexity	Notes
vKey = graph.key(v)	Get vertex key	V_i	
visited[vKey] = true	Mark as visited	V_i	
component.add(v)		V_i	
FOR EACH neighbor IN graph.adjVertices(v)	Iterate edges	E_i	
IF visited[graph.key(neighbor)] IS false	Check condition	E_i	
CALL dfs(graph, neighbor, visited, component)		E_i	
Total		$3V_i + 3E_i$	
Big(O)		$O(V_i + E_i)$	

3 DFS Time Complexity Analysis

Explanation

What is the running time of DFS? The loops on lines 1-3 and lines 5-7 of DFS take time (V) , exclusive of the time to execute the calls to DFS-Visit. As we did for breadth-first search, we use aggregate analysis. The procedure DFS-Visit is called exactly once for each vertex $v \in V$, since the vertex u on which DFS-Visit is invoked must be white and the first thing DFS-Visit does is paint vertex u gray. During an execution of DFS-Visit(G, v), the loop on lines 4-7 executes $|Adj[v]|$ times. Since

$$\sum_{v \in V} |Adj[v]| = \Theta(E)$$

the total cost of executing lines 4-7 of DFS-Visit is (E) . The running time of DFS is therefore $(V + E)$.

Key Points:

- **Initialization phase:** (V) - marking all vertices as unvisited
- **Main DFS loop:** (V) - checking each vertex for visitation status
- **DFS-Visit calls:** (E) - processing all edges across all vertices
- **Total complexity:** $(V + E)$ - additive combination of vertex and edge processing

(Source: Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to Algorithms (3rd ed.). MIT Press.)

Mathematical Basis:

- Each vertex visited exactly once $\rightarrow (V)$
- Each edge examined exactly once $\rightarrow (E)$
- Sum of all adjacency list sizes = $2E$ (undirected) or E (directed)

Application to Our Implementation:

Our `DFSFinishTime` follows this exact pattern with deterministic $(V + E)$ complexity, as it processes each vertex once and each edge once, regardless of graph structure.